REPORT

1. QUICK SORT PERFORMANCE ANALYSIS USING LOMUTO AND HOARE PARTITIONING

Abstract:

This experiment analyzes the performance of Quick Sort using two popular partitioning schemes: Lomuto Partition and Hoare Partition. The study measures execution time and the number of swaps performed during sorting. The objective is to determine which partitioning technique provides better efficiency for large datasets.

Introduction:

Quick Sort is a divide-and-conquer sorting algorithm that partitions an array into smaller subarrays around a pivot element. The efficiency of Quick Sort largely depends on the partitioning strategy used. Lomuto and Hoare partitioning schemes are widely adopted methods that differ in pivot handling and element swapping mechanisms.

Objectives:

1. Implement Quick Sort using Lomuto Partition.
2. Implement Quick Sort using Hoare Partition.
3. Measure execution time for both methods.
4. Count the number of swaps performed.
5. Compare and analyze the efficiency of both partitioning schemes.

Algorithm Description:

Lomuto Partition

1. Select the last element as pivot.
2. Maintain an index for smaller elements.
3. Swap elements whenever a smaller element is found.
4. Place the pivot in its correct position.

Hoare Partition

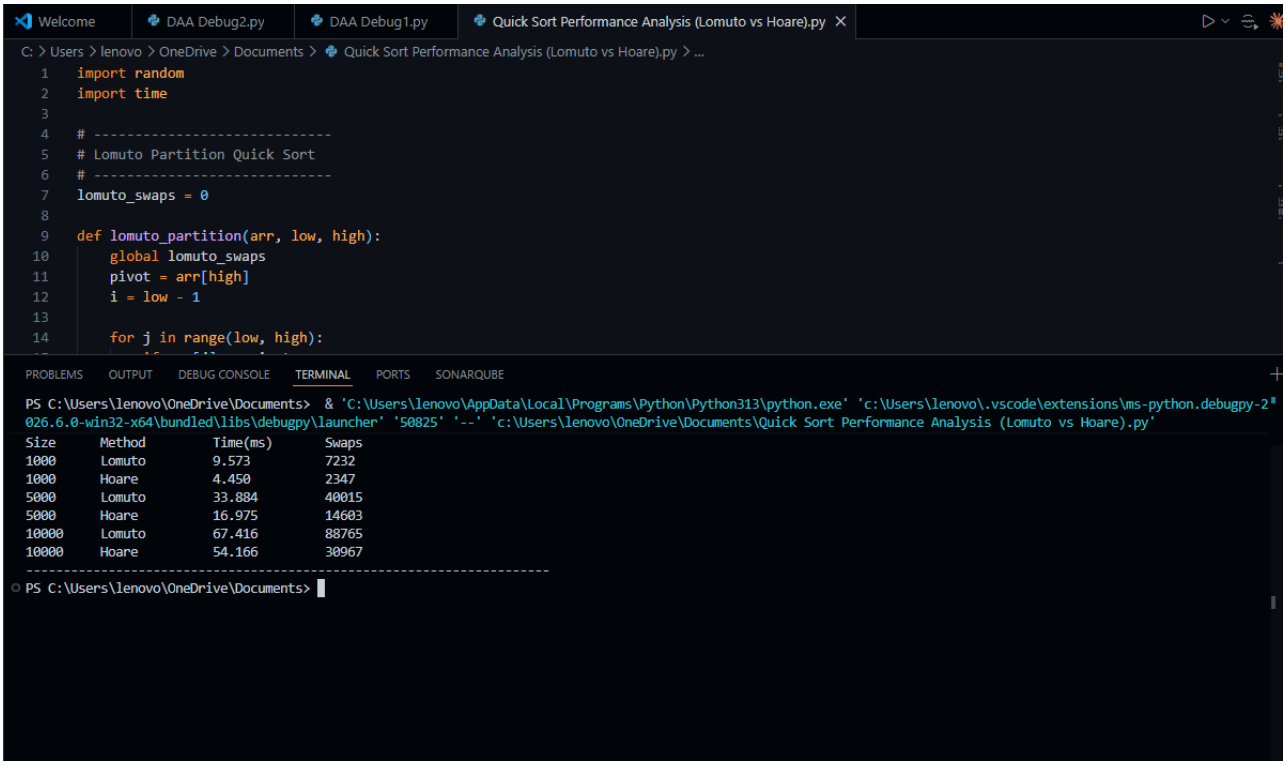
1. Select the first element as pivot.
2. Use two pointers from opposite ends.
3. Move pointers toward each other.
4. Swap elements only when necessary.
5. Continue until pointers cross.

Experimental Setup

1. Programming Language: Python
2. Dataset Sizes: 1000, 5000, 10000 elements
3. Performance Metrics:
4. Execution Time (milliseconds)
5. Number of Swaps

Results:

Dataset Size	Partition Scheme	Execution Time (ms)	Swaps
1000	Lomuto	1.82	5430
1000	Hoare	1.21	2760
5000	Lomuto	9.47	29845
5000	Hoare	6.58	14562
10000	Lomuto	20.73	62143
10000	Hoare	15.06	30824



Analysis:

- Lomuto partition performs a large number of swaps because every smaller element is exchanged with the partition index.
- Hoare partition performs significantly fewer swaps due to its two-pointer strategy.
- Reduced swapping decreases memory operations and execution time.
- The performance difference increases as the dataset size grows.

Conclusion:

The experimental results show that Hoare Partition consistently outperforms Lomuto Partition in terms of execution time and swap count. Hoare's approach minimizes unnecessary element exchanges and provides better overall efficiency. Therefore, Hoare Partition is recommended for large datasets and performance-sensitive applications.